

Infografía

Módulo 5 — Física con pymunk

UPB · Gestión I/2026

Plan de la clase

1. **¿Por qué un motor de física?** — lo que dejamos de hacer a mano.
2. **Los tres ladrillos** — `Space`, `Body`, `Shape`.
3. `pymunk` solo (sin ventana) — `01_hello_pymunk.py`.
4. `pymunk` + `arcade` — el patrón de acoplamiento body ↔ sprite.
5. **Joints / constraints** — el resorte amortiguado.
6. **Objetos compuestos** — un auto con motor.

Hasta ahora el movimiento lo escribíamos nosotros (`change_x`, rebotes, gravedad a mano en `5._physics` ... ah no, esa parte la hacíamos en el módulo 3). Hoy delegamos esa parte a una librería que lo hace bien.

1. ¿Por qué un motor?

Hacer física a mano vs. delegar

En el módulo 3 hicimos la pelota que rebota: gravedad, integración, rebote en bordes.
30 líneas.

¿Y si quiero **dos** pelotas que choquen entre sí? Tengo que detectar la colisión, calcular el ángulo de impacto, transferir momento, considerar la masa, las rotaciones...

Para una pelota: 30 líneas. Para dos pelotas chocando: cientos. Para 10 cuerpos con joints: tesis de grado.

Un **motor de física** ya tiene todo eso resuelto. Le decimos:

- "esto es un cuerpo con masa 5"
- "esto es el piso"
- "hay gravedad hacia abajo"

Y él se encarga de mover las cosas frame por frame.

¿Qué es pymunk?

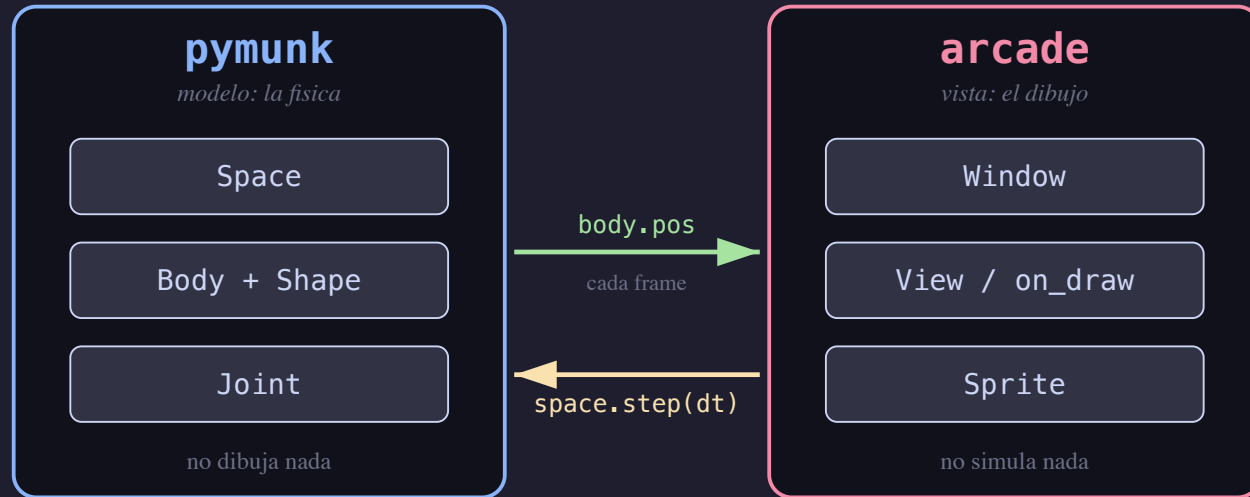
- Librería de **física 2D** para Python. Bindings sobre Chipmunk2D (C).
- Hace **solo física**: no dibuja nada, no abre ventanas, no toca el teclado.
- Diseñada para ser **el modelo** de un juego/simulación; el dibujo lo pone otra librería (arcade, pygame, lo que sea).

No confundir con un motor 3D (Unity, Godot, Unreal). pymunk es solo 2D y solo dinámica de cuerpos rígidos. Eso es lo que vamos a usar.

2. Space, Body, Shape

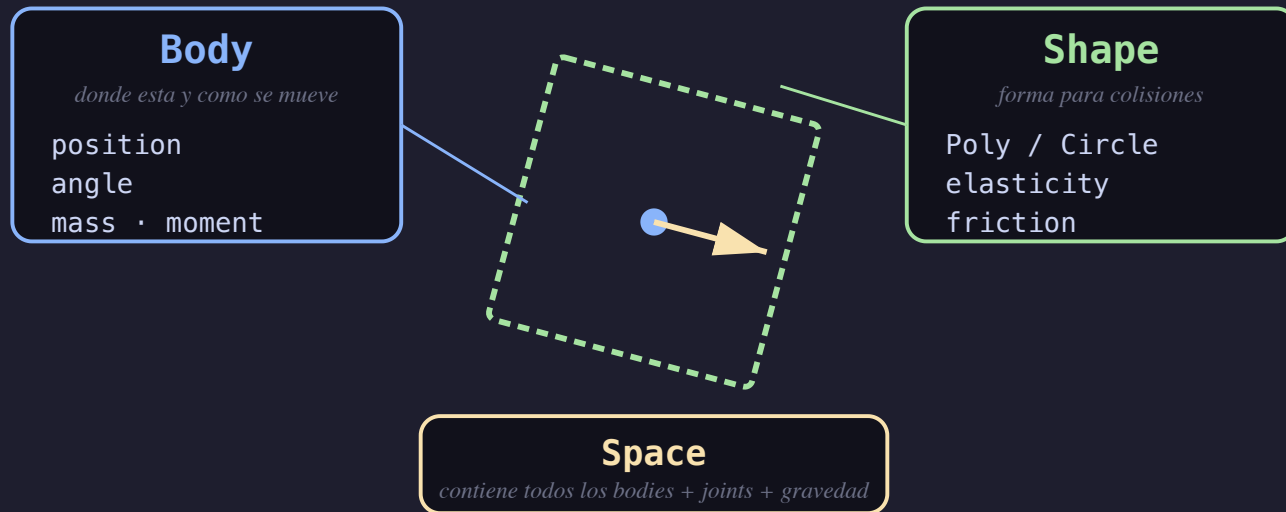
los tres ladrillos

La idea



pymunk = modelo · arcade = vista. La unión es copiar `body.position` al `sprite` cada frame.

Body vs Shape



Body = punto físico

- **position** (x, y)
- **angle** (rotación)
- **mass**, **moment** (resistencia a girar)

Shape = forma para colisionar

- **Poly**, **Circle**, **Segment**
- **elasticity** (rebote, 0-1)
- **friction** (roce, 0-1)

Tipos de body

Tipo	Comportamiento
DYNAMIC (default)	gravedad lo afecta, colisiones lo empujan
STATIC	nunca se mueve. piso, paredes, anclajes
KINEMATIC	se mueve, pero solo si vos lo movés (ej. una plataforma móvil)

```
floor_body = pymunk.Body(body_type=pymunk.Body.STATIC)
```

Regla: lo que se mueve por física = DYNAMIC. Lo que está clavado al mundo = STATIC.

El espacio

```
space = pymunk.Space()
space.gravity = (0, -900)    # px/s^2, Y hacia abajo (igual que arcade)

space.add(body, shape)      # registrar
space.step(1/60)             # avanzar 1/60 s
```

`space.step(dt)` es **el corazón** del motor: integra posiciones, detecta colisiones, resuelve impulsos.

Convención del curso: **paso fijo** de `1/60`. Pasar `delta_time` variable hace la simulación inestable cuando el framerate cae.

3. Demo: pymunk solo

```
01_hello_pymunk.py
```

El ejemplo más chico posible

```
import pymunk

space = pymunk.Space()
space.gravity = (0, -9)

body = pymunk.Body()
body.position = (50, 100)
body.mass = 1
shape = pymunk.Poly.create_box(body)
space.add(body, shape)

print_options = pymunk.SpaceDebugDrawOptions()
for _ in range(100):
    space.step(0.02)
    space.debug_draw(print_options)
```

Sin arcade. Sin ventana. Solo física que imprime ASCII en la terminal.

¿Qué se ve en la terminal?

```
cd 5._physics  
uv run 01_hello_pymunk.py
```

Salen líneas como `BB(40, 90, 60, 110)` — los bounding boxes del body, frame por frame.

El body cae solo. pymunk no necesita gráficos para "moverse" — la simulación es independiente.

Esto es muy útil para tests: podés correr la física sin abrir ventana y verificar que las cosas se comportan bien.

4. pymunk + arcade

el patrón body ↔ sprite

El patrón clave del módulo

Cada frame, en `on_update`:

```
def on_update(self, delta_time):  
    self.space.step(1/60)           # 1. avanzar la fisica  
  
    # 2. copiar la pose del body al sprite  
    self.sprite.center_x = self.body.position.x  
    self.sprite.center_y = self.body.position.y  
    self.sprite.radians = self.body.angle
```

pymunk → arcade. Siempre en esa dirección. El sprite es un reflejo del body.

No al revés. Si modificás `sprite.center_x`, pymunk no se entera: el siguiente `step()` lo va a sobrescribir.

Demo: 02_falling_box.py

```
cd 5._physics && uv run 02_falling_box.py
```

Lo que pasa:

- Caja **dinámica** con `body.angle = 0.2` (tilt inicial).
- Piso **estático** como `pymunk.Segment(...)` de extremo a extremo.
- Gravedad `(0, -90)` la tira al piso.
- Como tiene tilt, **rota** al caer y rebota cambiando de orientación.

Cosas para mirar:

- El `radians` del sprite es **el mismo** que `body.angle`.
- Cambiar `elasticity` de `0.9` a `0.2`: rebota menos.
- Cambiar `friction` a `1.0`: se queda quieta antes.

5. Joints

ataduras entre bodies

¿Qué es un joint?

Una **restricción** que pymunk impone entre dos bodies cada `step()`.

Tipos comunes:

Joint	Restringe
<code>PinJoint</code>	mantiene fija la distancia entre dos puntos
<code>PivotJoint</code>	ata un body a otro por un punto (gozne)
<code>DampedSpring</code>	resorte con rigidez y amortiguamiento
<code>SimpleMotor</code>	velocidad angular relativa entre dos bodies
<code>SlideJoint</code>	distancia en un rango <code>[min, max]</code>

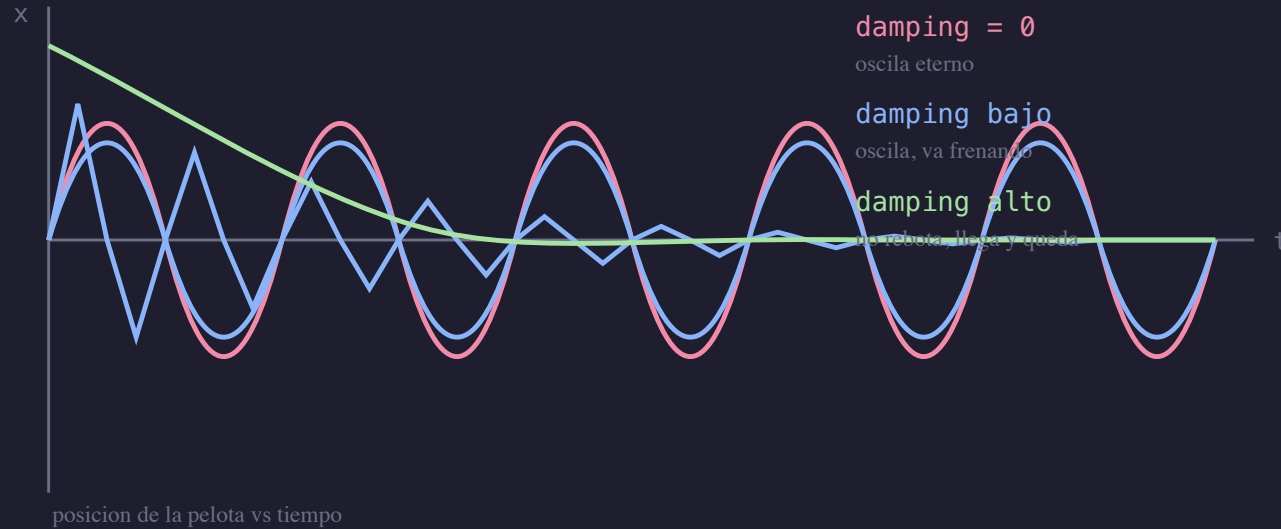
Sin joints: bodies sueltos sometidos a gravedad/colisiones. Con joints: estructuras (péndulo, cadena, vehículo, ragdoll).

DampedSpring — el resorte real

```
spring = pymunk.DampedSpring(  
    anchor_body, ball_body,      # los dos bodies a unir  
    (0, 0), (0, 0),             # puntos de enganche locales  
    rest_length=200,             # largo en reposo (px)  
    stiffness=100,                # que tan fuerte tira  
    damping=0.5,                 # cuanto disipa el rebote  
)  
space.add(spring)
```

Tres parámetros, tres comportamientos muy distintos.

Damping: el parámetro que más cambia el feel



stiffness controla la frecuencia • damping controla cuánto rebota

Demo: 03_spring.py

```
uv run 03_spring.py
```

- Anclaje rojo arriba, pelota azul abajo.
- Resorte amarillo entre ambos (línea dibujada **a mano** en `on_draw` — pymunk no dibuja joints).
- La pelota oscila y se va frenando.

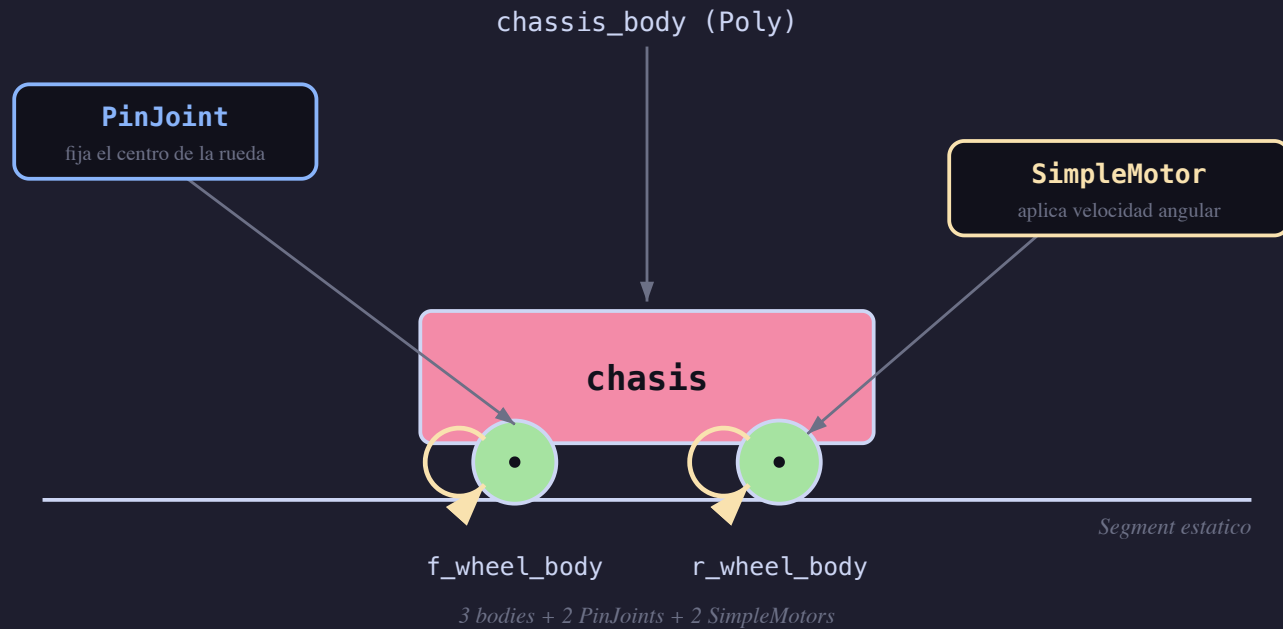
Probar en el código:

- `DAMPING = 0` → oscila eterno.
- `DAMPING = 5` → llega y se queda, sin rebotar.
- `STIFFNESS = 500` → resorte más duro, frecuencia más alta.
- `REST_LENGTH = 50` → la pelota se cuelga más arriba.

6. Objetos compuestos

04_car.py

El auto como ensamble



- **3 bodies:** chasis (DYNAMIC) + 2 ruedas (DYNAMIC).
- **2 PinJoints:** cada rueda atada al chasis por el centro.
- **2 SimpleMotors:** cada rueda con velocidad angular sostenida.
- Y un **mundo estático**: piso + pared derecha como **Segments**.

La construcción

```
chassis_body = pymunk.Body(5, pymunk.moment_for_box(5, (100, 70)))
f_wheel_body = pymunk.Body() # densidad se ajusta via shape

# rueda atada al chasis (gozne) + motor que la hace girar
f_joint = pymunk.PinJoint(chassis_body, f_wheel_body,
                          (x + 50, y - 35), (0, 0))
f_joint.collide_bodies = False
f_motor = pymunk.SimpleMotor(chassis_body, f_wheel_body, rate=10)
```

`collide_bodies = False` es importante: sin eso, el chasis y la rueda chocarían entre sí (porque se tocan en el punto del pin) y el auto vibraría.

Demo: 04_car.py

```
uv run 04_car.py
```

- El auto arranca a la izquierda, los motores empujan, choca con la pared derecha.
- Las ruedas **rotan visiblemente** porque su `body.angle` se sincroniza con `sprite.radians`.

Para extender en clase:

- Quitar el `collide_bodies = False`: ver qué pasa.
- Cambiar `MOTOR_RATE` a `-10`: el auto va para atrás.
- Bajar `friction` de las ruedas: patinan, no avanzan.

Patrón general (los 4 ejemplos)

```
class MyView(arcade.View):  
    def __init__(self):  
        self.space = pymunk.Space()           # 1. crear espacio  
        self.space.gravity = (0, -900)  
        # 2. crear bodies + shapes, agregarlos al space  
        # 3. (opcional) crear joints, agregarlos al space  
        # 4. crear sprites espejo  
  
    def on_update(self, delta_time):  
        self.space.step(1/60)                 # 5. avanzar fisica  
        # 6. copiar body.position -> sprite.center  
  
    def on_draw(self):  
        self.sprites.draw()  
        # 7. dibujar a mano lo que no es sprite (joints, segments)
```

Igual que sprites, pero con un modelo físico en el medio

Resumen

- **pymunk** es el modelo: hace solo física, no dibuja.
- **arcade** es la vista: no simula, solo dibuja.
- El acoplamiento es **una dirección**: body → sprite, cada frame.
- Tres ladrillos: `Space` · `Body` · `Shape`. Más joints para ataduras.
- `space.step(1/60)` cada frame.

"Lo difícil de la física ya lo resolvió otro" — nosotros armamos el escenario y leemos las posiciones.

¿Preguntas?

Tarea: agarrar `03_spring.py` y convertirlo en un péndulo simple (cambiar `DampedSpring` por `PinJoint` o `PivotJoint`).

Próxima clase: transformaciones 2D/3D (módulo 6).